

CMU · 14-455 | Database System (2019)

CMU 14-455 (2019) · 课程资料包 @ShowMeAI



视频

中英双语字幕



课件

一键打包下载



笔记

官方笔记翻译



代码

作业项目解析



视频 · B 站 [扫码或点击链接]

<https://www.bilibili.com/video/BV1qf4y1J7mX>



课件 & 代码 · 博客 [扫码或点击链接]

<http://blog.showmeai.tech/cmu-14-455/>

高级 SQL

聚合

排序

并发控制

数据库

内存管理

查询规划

分组

树索引

索引构建

查询执行

Awesome AI Courses Notes Cheatsheets 是 [ShowMeAI](#) 资料库的分支系列，覆盖最具知名度的 **TOP50+** 门 AI 课程，旨在为读者和学习者提供一整套高品质中文学习笔记和速查表。

点击课程名称，跳转至课程**资料包**页面，**一键下载**课程全部资料！

机器学习	深度学习	自然语言处理	计算机视觉
Stanford · CS229	Stanford · CS230	Stanford · CS224n	Stanford · CS231n
# Awesome AI Courses Notes Cheatsheets · 持续更新中			
知识图谱	图机器学习	深度强化学习	自动驾驶
Stanford · CS520	Stanford · CS224W	UCBerkeley · CS285	MIT · 6.S094



微信公众号

资料下载方式 2: 扫码点击**底部菜单栏**
称为 **AI 内容创作者?** 回复 [添砖加瓦]

Lecture #19: Multi-Version Concurrency Control

15-445/645 Database Systems (Fall 2019)

<https://15445.courses.cs.cmu.edu/fall2019/>

Carnegie Mellon University

Prof. Andy Pavlo

1 Multi-Version Concurrency Control

Multi-Version Concurrency Control (MVCC) is a larger concept than just a concurrency control protocol. It involves all aspect of the DBMS's design and implementation. MVCC is the most widely used scheme in DBMS. It is now used in almost every new DBMS implemented in last 10 years. Even some systems (e.g., NoSQL) that do not support multi-statement transactions use it.

The DBMS maintains multiple physical versions of a single logical object in the database.

- When a transaction writes to an object, the DBMS creates a new version of that object.
- When a transaction reads an object, it reads the newest version that existed when the transaction started.

The original MVCC protocol for DBMSs was first proposed in 1978 MIT PhD dissertation. The first implementation in a real DBMS was in InterBase (now open-sourced as Firebird) by Jim Starkey, who later went on to be the co-founder of NuoDB.

Key Properties

Writers don't block the readers. Readers don't block the writers.

Read-only transactions can read a consistent **snapshot** without acquiring locks. Timestamps are used to determine visibility.

Multi-versioned DBMSs can support *time-travel queries* that can read the database at a point-in-time snapshot.

There are four important MVCC design decisions:

1. Concurrency Control Protocol (T/O, OCC, 2PL, etc).
2. Version Storage
3. Garbage collection
4. Index Management

2 Version Storage

This how the DBMS will store the different physical versions of a logical object.

The DBMS uses the tuple's pointer field to create a **version chain** per logical tuple. This allows the DBMS to find the version that is visible to a particular transaction at runtime. Indexes always point to the head of the chain. A thread traverses chain until you find the version that is visible to you. Different storage schemes determine where/what to store for each version.

Approach #1: Append-Only Storage – New versions are appended to the same table space.

- **Oldest-To-Newest (O2N):** Append new version to end of chain, look-ups require entire chain traversal.
- **Newest-To-Oldest (N2O):** Head of chain is newest, look-ups are quick, but indexes need to be updated every version.

Approach #2: Time-Travel Storage – Old versions are copied to separate table space.

Approach #3: Delta Storage – The original values of the modified attributes are copied into a separate delta record space.

3 Garbage Collection

The DBMS needs to remove **reclaimable** physical versions from the database over time.

Approach #1: Tuple Level Garbage Collection – Find old versions by examining tuples directly

- **Background Vacuuming:** Separate threads periodically scan the table and look for reclaimable versions, works with any version storage scheme.
- **Cooperative Cleaning:** Worker threads identify reclaimable versions as they traverse version chain. Only works with O2N.

Approach #2: Transaction Level – Each transaction keeps track of its own read/write set. When a transaction completes, the garbage collector can use that to identify what tuples to reclaim. The DBMS determines when all versions created by a finished transaction are no longer visible.

4 Index Management

All primary key (pkey) indexes always point to version chain head. How often the DBMS has to update the pkey index depends on whether the system creates new versions when a tuple is updated. If a transaction updates a pkey attribute(s), then this is treated as a DELETE followed by an INSERT.

Managing secondary indexes is more complicated:

- **Approach #1: Logical Pointers** – Use a fixed identifier per tuple that does not change. Requires an extra indirection layer that maps the logical id to the physical location of the tuple (Primary Key vs Tuple ID).
- **Approach #2: Physical Pointers** – Use the physical address to the version chain head

CMU · 14-455 | Database System (2019)

CMU 14-455 (2019) · 课程资料包 @ShowMeAI



视频

中英双语字幕



课件

一键打包下载



笔记

官方笔记翻译



代码

作业项目解析



视频 · B 站 [扫码或点击链接]

<https://www.bilibili.com/video/BV1qf4y1J7mX>



课件 & 代码 · 博客 [扫码或点击链接]

<http://blog.showmeai.tech/cmu-14-455/>

高级 SQL

聚合

排序

并发控制

数据库

内存管理

查询规划

分组

树索引

索引构建

查询执行

Awesome AI Courses Notes Cheatsheets 是 [ShowMeAI](#) 资料库的分支系列，覆盖最具知名度的 **TOP50+** 门 AI 课程，旨在为读者和学习者提供一整套高品质中文学习笔记和速查表。

点击课程名称，跳转至课程**资料包**页面，**一键下载**课程全部资料！

机器学习	深度学习	自然语言处理	计算机视觉
Stanford · CS229	Stanford · CS230	Stanford · CS224n	Stanford · CS231n
# Awesome AI Courses Notes Cheatsheets · 持续更新中			
知识图谱	图机器学习	深度强化学习	自动驾驶
Stanford · CS520	Stanford · CS224W	UCBerkeley · CS285	MIT · 6.S094



微信公众号

资料下载方式 2: 扫码点击**底部菜单栏**称为 **AI 内容创作者?** 回复 [添砖加瓦]